> Neka je dat realni polinom $P(x)$ nad realnim segmentom $[a,b]$.
>
> 1. A. Odrediti Euklidovim algoritmom najveći zajednički delilac $G(x) = GCD(P(x),P'(x))$.
>   B. Za polinom $P(x):=P(x)/G(x)$, upotrebom Šturmove teoreme, odrediti broj nula na $[a,b]$.
> 2. Primena Šturmove teoreme.
>
>   Neka je k broj decimala na koji se zaokružuje neki realan broj. Za polinom
>
> $P(x) = a_n x^n + a_{n-1}x^{n-1} + ... + a_1 x + a_0$, sa realnim koeficijentima
>
> $a_n, a_{n-1}, ..., a_1, a_0$
>
>   odrediti niz naniže zaokruženih racionalnih koeficijenata
>
> $\alpha_n, \alpha_{n-1}, ..., \alpha_1, \alpha_0$
>
>   određenih po sledećim pravilima:
>
>   - ako je $a_k = a_0.a_1...a_k a_{k+1} > 0$ tada $\alpha_k = a_0.a_1...a_k$
>   - ako je $a_k = a_0.a_1...a_k a_{k+1} < 0$ tada $\alpha_k = a_0.a_1...a_k'$, gdje je a_k' naviše zaokružena cifra (uz eventualno posledično zaokruživanje i prethodnih cifara za jedan broj naviše).
>
>   Odrediti proceduru za formiranje racionalnog polinoma
>
> $P(x) = \alpha_n x^n + \alpha_{n-1}x^{n-1} + ... + \alpha_1 x + \alpha_0$.
>
>   Neka je $[a,b]$ segment sa racionalnim rubnim tačkama. Naći takav polinom $P(x)$ sa realnim koecijentima i broj k, da na osnovu pozitivnosti polinoma $P(x)$ nad segmentom $[a,b]$ imamo dokaz o pozitivnosti polinoma $P(x)$ nad segmentom $[a,b]$.

Napisani kod ćemo testirati na primjeru polinoma $P(x) = x^9 - 3x^7 - x^6 + 3x^5 + 3x^4 - x^3 - 3x^2 + 1$ i nule ćemo tražiti na segmentu $[0,3]$.

In [95]:
```
from numpy.polynomial.polynomial import Polynomial

a, b = 0, 3
p = Polynomial([1, 0, -3, -1, 3, 3, -1, -3, 0, 1]) # x^9 - 3x^7 - x^6 + 3x^5 + 3x^4 - x^3 - 3x^2 + 1
print(f"Ulazni polinom: {p}")
```

```
Ulazni polinom: 1.0 + 0.0 x - 3.0 x**2 - 1.0 x**3 + 3.0 x**4 + 3.0 x**5 - 1.0 x**6 -
3.0 x**7 + 0.0 x**8 + 1.0 x**9
```

Prvi korak je uzimanje izvoda polinoma i traženje NZD-a. NZD računamo u funkciji *polynomial_gcd*, koristeći Euklidov algoritam. Dok za računanje izvoda se koristi postojeća funkcija u *numpy* biblioteci.

Zbog numeričkih grešaka prilikom rada sa realnim brojevima, može doći do situacije u kojoj je ostatak veoma mali, ali ne i nula. Zbog toga se u uslovu za nastavak dijeljenja poredi apsolutna vrednost koeficijenta uz najviši stepen polinoma.

In [96]:
```python
def polynomial_gcd(a: Polynomial, b: Polynomial):
        if len(b) > len(a):
                a, b = b, a

        while abs(b.coef[-1]) > 1e-10:
                r = a % b
                a, b = b, r

        return a / a.coef[-1]

g = polynomial_gcd(p, p.deriv())
print(f"NZD: {g}")
```

```
NZD: -1.0 + 1.0 x + 2.0 x**2 - 2.0 x**3 - 1.0 x**4 + 1.0 x**5
```

Krećemo od prvog polinoma u Šturmov-ovom nizu (*P(x) / G(x)*), i od njega gradimo naredne polinome. Primjenjujemo Šturmov algoritam za generisanje novih polinoma u lancu.

Kada smo generisali sve polinome, možemo preći na brojanje promjena znaka u početnoj i krajnjoj tački intervala. Razlika između ta dva broja predstavlja broj nula u intervalu. To nam pomaže da odredimo da li je polinom strogo pozitivan/negativan.

In [97]:
```python
def generate_sturms_chain(p: Polynomial):
        chain = [p, p.deriv()]

        while chain[-1].degree() > 0:
                chain.append(-(chain[-2] % chain[-1]))

        return chain

def sign_changes(chain: list[Polynomial], x: float):
```

```
        values = [p(x) for p in chain]

        changes = 0
        for i in range(1, len(values)):
                if values[i] * values[i - 1] < 0:
                        changes += 1

        return changes

def calculate_number_of_zeroes(p: Polynomial, scope: tuple[float, float]):
        a, b = scope

        chain = generate_sturms_chain(p)

        print("Lanac polinoma: ")
        for p in chain:
                print('\t' + str(p))

        sign_changes_a = sign_changes(chain, a)
        sign_changes_b = sign_changes(chain, b)

        print(f"Promjena znaka u '{a}': {sign_changes_a}")
        print(f"Promjena znaka u '{b}': {sign_changes_b}")

        return sign_changes_a - sign_changes_b

zeros = calculate_number_of_zeroes(p // g, (a, b))
print("1) Broj nula:", zeros)
```

```
Lanac polinoma:
        -1.0 - 1.0 x - (1.33226763e-15) x**2 + 1.0 x**3 + 1.0 x**4
        -1.0 - (2.66453526e-15) x + 3.0 x**2 + 4.0 x**3
        0.9375 + 0.75 x + 0.1875 x**2
        -64.0 - 32.0 x
        -0.1875
Promjena znaka u '0': 2
Promjena znaka u '3': 1
1) Broj nula: 1
```

Za naredni dio zadatka uzimamo sledeću polinomsku funkciju i parametre:

In [98]:
```
from math import pi

a, b = 0, 1.35
p = Polynomial([
        0, 0, 0, 0,
        2*pi/3 - 2,
        -(pi**2)/60 + pi/30,
        -pi/30 + 1/10,
        -(pi**2)/1680+pi/840,
        pi/1260-1/420
])
print(f"Ulazni polinom: {p}")
```

```
Ulazni polinom: 0.0 + 0.0 x + 0.0 x**2 + 0.0 x**3 + 0.0943951 x**4 - 0.05977365 x**5 -
0.00471976 x**6 - 0.00213477 x**7 + 0.00011238 x**8
```

Prvo je potrebno izvršiti zaokruživanje cifara po algoritmu datom u zadatku.

In [99]:
```
from math import floor, log10, ceil

def spec_round(x: float, k: int):
        if x == 0:
                return 0
        elif x > 0:
                exponent = floor(log10(x))
                mantissa = x / (10**exponent)

                mantissa = floor(mantissa * (10**k)) / (10**k)

                return mantissa * 10**exponent
        else:
                exponent = floor(log10(-x))
                mantissa = -x / (10**exponent)

                mantissa = ceil(mantissa * (10**k)) / (10**k)

                return -mantissa * 10**exponent

k = 2
```

```
p = Polynomial([spec_round(coef, k) for coef in p])
print(f"Sređeni polinom: {p}")
```

```
Sređeni polinom: 0.0 + 0.0 x + 0.0 x**2 + 0.0 x**3 + 0.0943 x**4 - 0.0598 x**5 -
0.00472 x**6 - 0.00214 x**7 + 0.000112 x**8
```

Gledajući vrijednosti funkcije na krajevima intervala, određujemo dozvoljeni broj nula na malo proširenom intervalu. Potom brojimo nule unutar intervala i utvrđujemo da li je funkcija strogo pozitivna.

In [100…

```
allowed_zeros = 0
if p(a) == 0:
        a -= 0.1
        allowed_zeros += 1
if p(b) == 0:
        b += 0.1
        allowed_zeros += 1

print(f"Dozvoljeni broj nula: {allowed_zeros}")
```

```
Dozvoljeni broj nula: 1
```

In [101…

```
zeros = calculate_number_of_zeroes(p, (a, b))
print(f"Broj nula na intervalu: {zeros}")

always_positive = zeros == allowed_zeros and p((a+b)/2) > 0
print(f"2) Polinom {'je' if always_positive else 'nije'} uvijek pozitivan")
```

```
Lanac polinoma:
        0.0 + 0.0 x + 0.0 x**2 + 0.0 x**3 + 0.0943 x**4 - 0.0598 x**5 -
0.00472 x**6 - 0.00214 x**7 + 0.000112 x**8
        0.0 + 0.0 x + 0.0 x**2 + 0.3772 x**3 - 0.299 x**4 - 0.02832 x**5 -
0.01498 x**6 + 0.000896 x**7
        -0.0 + -0.0 x + -0.0 x**2 - 0.11261272 x**3 + 0.04211618 x**4 +
0.03087991 x**5 + 0.00565227 x**6
        -0.0 + -0.0 x + -0.0 x**2 + 0.01878086 x**3 + 0.13305516 x**4 -
0.07358696 x**5
        0.0 + 0.0 x + 0.0 x**2 + 0.10212319 x**3 - 0.11787306 x**4
        -0.0 + -0.0 x + -0.0 x**2 - 0.07882179 x**3
        -0.0
Promjena znaka u '-0.1': 4
Promjena znaka u '1.35': 3
Broj nula na intervalu: 1
2) Polinom je uvijek pozitivan
```

S obzirom da je polinom uvijek pozitivan, možemo zaključiti i da je polazni polinom strogo pozitivan na datom intervalu. To implicira i da razmatrani polinom nema nula unutar istog intervala.